

Trabalho 1: Contagem de Operações e de Tempo de Execução

SCC0502 - Algoritmos e Estruturas de Dados I - ICMC/USP

Enzo Ribeiro Ferrari (nº USP 15574520)
Matheus de Souza Santana (nº USP 17839560)
Pietro Augusto Venancio da Costa (nº USP 17839104)
Matheus Bastos Machado (nº USP 18126601)

18 de setembro de 2026

Sumário

1	Introdução	2
2	Análise Empírica	3
2.1	Metodologia experimental	3
2.2	Contagem de operações no pior caso	4
2.3	Tempos médios de execução	5
2.4	Gráficos	6
2.5	Discussão dos resultados	11
3	Análise Assintótica Teórica	13
3.1	Modelo de contagem adotado	13
3.2	Inversão da ordem do vetor	13
3.3	Busca sequencial	14
3.4	Busca binária iterativa	16
3.5	Busca binária recursiva	17
3.6	Quadro-resumo	19
4	Conclusão	20
5	Contribuições dos integrantes	21
6	Considerações finais	21
6.1	Organização do projeto e repositório	21
6.2	Dificuldades encontradas	22

1 Introdução

Este trabalho compara quatro algoritmos elementares sobre vetores de inteiros, do ponto de vista do custo de execução:

1. **inversão da ordem do vetor;**
2. **busca sequencial;**
3. **busca binária iterativa;**
4. **busca binária recursiva.**

O problema que motiva a comparação é simples de enunciar e menos simples de responder com rigor: *quanto custa cada um desses algoritmos, e como esse custo cresce quando a entrada cresce?* Responder “o mais rápido é o melhor” seria insuficiente, porque a velocidade medida em segundos depende do computador, do compilador e até da carga do sistema no momento da medição. O que caracteriza um algoritmo é a ordem de crescimento do seu custo em função do tamanho da entrada.

Para investigar isso, o trabalho foi conduzido em três frentes que se conferem mutuamente:

- **Implementação.** Os quatro algoritmos foram escritos em C, sem bibliotecas externas, e integrados a um programa com menu que lê um vetor ordenado e executa comandos até receber o comando de encerramento.
- **Análise empírica.** Para cada algoritmo e para cada um dos cinco tamanhos exigidos (1.000, 50.000, 100.000, 500.000 e 1.000.000 de elementos), foram medidos o tempo médio de execução e as quantidades de comparações e atribuições no pior caso. As medições foram feitas por programas auxiliares separados do programa principal.
- **Análise assintótica teórica.** Para cada algoritmo foi deduzida a função de custo $T(n)$ pela contagem dos passos executados, no melhor caso, no caso médio e no pior caso. De cada função foi identificado o termo dominante e obtida a classificação em notação O .

O resultado dessas três frentes é conferido cruzando as contagens previstas pelas fórmulas com as contagens efetivamente medidas pelos programas instrumentados. Quando os dois lados coincidem, temos evidência de que a implementação, a instrumentação e a dedução matemática estão de acordo.

Uma decisão metodológica atravessa todo o trabalho e convém deixá-la explícita desde já: contagem de operações e tempo de execução são duas medidas diferentes, e não devem ser confundidas. A contagem é determinística, reproduz-se em qualquer máquina e é a base da classificação assintótica. O tempo é uma medida física, sujeita a ruído, e serve para verificar se o comportamento previsto aparece na prática. Boa parte da discussão deste relatório está exatamente nos pontos em que as duas medidas não andam juntas.

2 Análise Empírica

2.1 Metodologia experimental

Todas as medições seguiram o mesmo protocolo, aplicado aos quatro algoritmos.

2.1.1 Entrada

Para cada tamanho n , o vetor é alocado dinamicamente e preenchido com os valores $0, 1, 2, \dots, n-1$. Essa entrada é ordenada, condição necessária para as duas buscas binárias, e é usada também nos outros dois algoritmos para que a comparação entre eles seja feita sobre a mesma entrada.

2.1.2 Definição do pior caso

Nas três buscas, o pior caso é obtido procurando $p = n$. Como o vetor contém apenas os valores de 0 a $n-1$, esse valor não existe:

- **Busca sequencial.** É obrigada a examinar todas as n posições antes de concluir que o valor está ausente.
- **Buscas binárias.** Encontram sempre $v[\text{meio}] < p$, seguem sempre para a metade direita, que é a maior das duas com o cálculo de meio usado, e só param quando o intervalo fica vazio, percorrendo o caminho mais longo possível.

A inversão não tem pior caso distinto: para um mesmo n , ela executa a mesma quantidade de operações qualquer que seja o conteúdo do vetor. Isso é justificado na Seção 3.2.

2.1.3 Separação entre tempo e contagem

Este é o ponto central do protocolo. Cada experimento usa duas funções:

- **Função original.** A mesma usada pelo programa principal, sem contadores, empregada exclusivamente para medir tempo.
- **Função instrumentada.** Definida no programa auxiliar, com a mesma lógica acrescida dos contadores, empregada exclusivamente para contar operações.

Se os contadores estivessem dentro da função cronometrada, o tempo medido incluiria o custo de incrementar os contadores, e não o custo do algoritmo. A versão instrumentada é executada uma única vez por tamanho, fora da região cronometrada: como a contagem é determinística para um mesmo n e um mesmo valor procurado, uma execução basta.

2.1.4 Medição do tempo

Uma única execução de qualquer um destes algoritmos é curta demais para a resolução prática de `clock()`. Por isso, cada medição cronometra um bloco de repetições e divide o tempo total pelo número de repetições. O enunciado exige no mínimo 100 execuções por entrada; todas as configurações usaram esse mínimo ou mais:

Experimento	Regra de repetições	Repetições por tamanho
Inversão	$\max(100, 10^8/n)$	100.000 / 2.000 / 1.000 / 200 / 100
Busca sequencial	$\max(100, 10^8/n)$	100.000 / 2.000 / 1.000 / 200 / 100
Busca binária iterativa	fixo	1.000.000
Busca binária recursiva	fixo	1.000.000

Os dois algoritmos lineares usam a mesma regra porque seu custo cresce proporcionalmente a n : entradas maiores precisam de menos repetições para formar um bloco mensurável. As duas buscas binárias usam o mesmo valor fixo e alto, porque seu custo quase não cresce com n e uma busca isolada leva dezenas de nanossegundos. O valor ser o mesmo nas duas é intencional: é isso que permite compará-las diretamente.

Ficam fora da região cronometrada a alocação, o preenchimento do vetor, o aquecimento, a contagem de operações, as validações, os `printf` e a gravação dos resultados. Ficam dentro apenas o laço de repetições e as chamadas à função original. Antes de cada medição são feitas execuções de aquecimento (10 nos experimentos lineares, 100 nos de busca binária), descartadas, para reduzir o peso do primeiro acesso ao código e aos dados.

Nas buscas, o retorno é armazenado em uma variável `volatile`, para reduzir a chance de o compilador eliminar a chamada por considerar seu resultado desnecessário. Os programas auxiliares são compilados com `-O0` e ligados separadamente às funções originais, sem otimização entre módulos.

2.1.5 Validação automática

Os quatro programas auxiliares comparam as contagens medidas com as fórmulas teóricas e emitem aviso em caso de divergência. Os três que executam buscas verificam também se o retorno é o esperado para o pior caso, que é sempre “valor ausente”. Nenhum aviso foi emitido nesta coleta.

2.2 Contagem de operações no pior caso

As contagens abaixo foram medidas pelas versões instrumentadas e referem-se a uma execução de cada algoritmo. Elas não são multiplicadas pelo número de repetições usado na medição de tempo.

Comparações e atribuições são contadas separadamente, em três tabelas. Na Seção 3 elas aparecem como $C(n)$, o número de comparações, e $A(n)$, o número de atribuições; a convenção que define o que entra em cada uma está na Seção 3.1.

2.2.1 Comparações em uma execução

n	Inversão	Sequencial	Bin. iterativa	Bin. recursiva
1.000	501	2.001	31	31
50.000	25.001	100.001	49	49
100.000	50.001	200.001	52	52
500.000	250.001	1.000.001	58	58
1.000.000	500.001	2.000.001	61	61

2.2.2 Atribuições em uma execução

n	Inversão	Sequencial	Bin. iterativa	Bin. recursiva
1.000	2.502	1.001	22	10
50.000	125.002	50.001	34	16
100.000	250.002	100.001	36	17
500.000	1.250.002	500.001	40	19
1.000.000	2.500.002	1.000.001	42	20

2.2.3 Custo total no modelo adotado

Somando as duas tabelas anteriores, obtemos $T(n) = C(n) + A(n)$:

n	Inversão	Sequencial	Bin. iterativa	Bin. recursiva
1.000	3.003	3.002	53	41
50.000	150.003	150.002	83	65
100.000	300.003	300.002	88	69
500.000	1.500.003	1.500.002	98	77
1.000.000	3.000.003	3.000.002	103	81

Todas essas contagens coincidem com as fórmulas deduzidas na Seção 3, para todos os tamanhos.

2.3 Tempos médios de execução

Cada coluna usa a unidade mais adequada à sua faixa de valores: microssegundos (μs , ou 10^{-6} s) para os dois algoritmos lineares e nanossegundos (ns, ou 10^{-9} s) para as duas buscas binárias. Um microssegundo equivale a mil nanossegundos.

n	Inversão (μs)	Sequencial (μs)	Bin. iterativa (ns)	Bin. recursiva (ns)
1.000	0,60	0,45	13	18
50.000	29,00	22,50	23	30
100.000	58,00	44,00	25	31
500.000	290,00	225,00	28	34
1.000.000	580,00	440,00	27	37

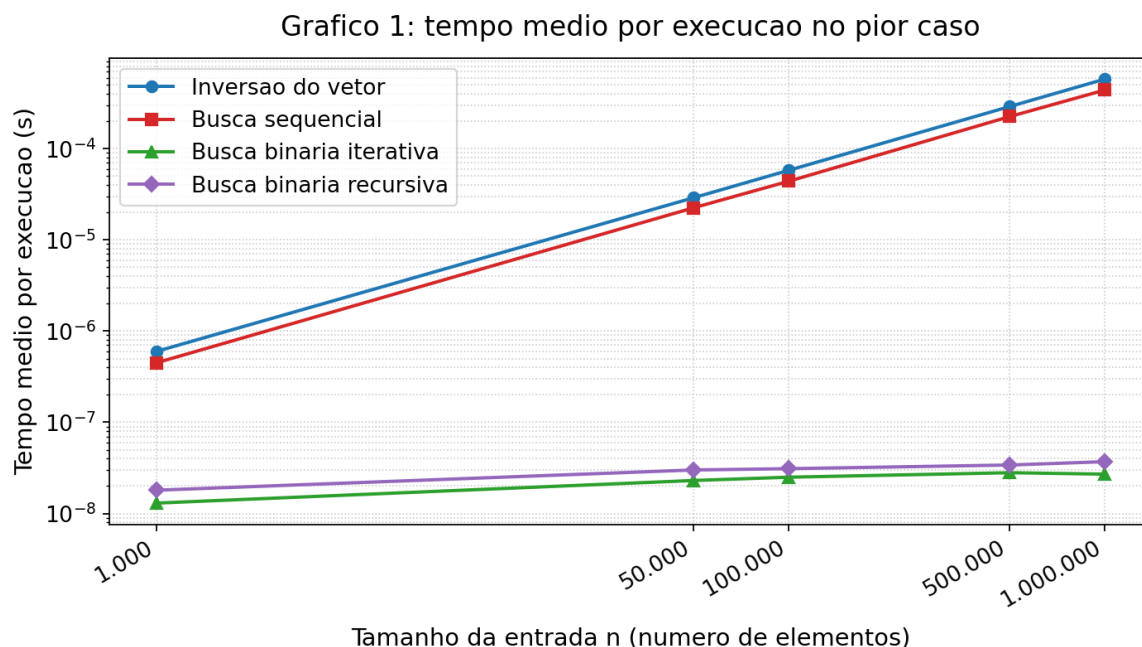
As duas faixas ficam a quatro ordens de grandeza de distância. Em um vetor de um milhão de elementos, a busca sequencial leva cerca de 440 μs , o equivalente a aproximadamente 0,44 milissegundo, enquanto a busca binária iterativa leva cerca de 27 ns. A razão entre os dois tempos é de aproximadamente 16.300 vezes.

Os valores das duas buscas binárias aparecem sem casas decimais porque são o resultado de dividir o tempo do bloco, medido em milissegundos inteiros, por um milhão de repetições. A resolução do relógio nesse computador é de um milissegundo, e casas decimais ali sugeririam uma precisão que a medição não tem.

2.4 Gráficos

Esta seção traz sete gráficos. Os Gráficos 1 a 3 comparam os quatro algoritmos entre si; o Gráfico 4 isola os dois algoritmos lineares; os Gráficos 5 a 7 isolam as duas buscas binárias. O comentário de cada gráfico vem logo abaixo dele.

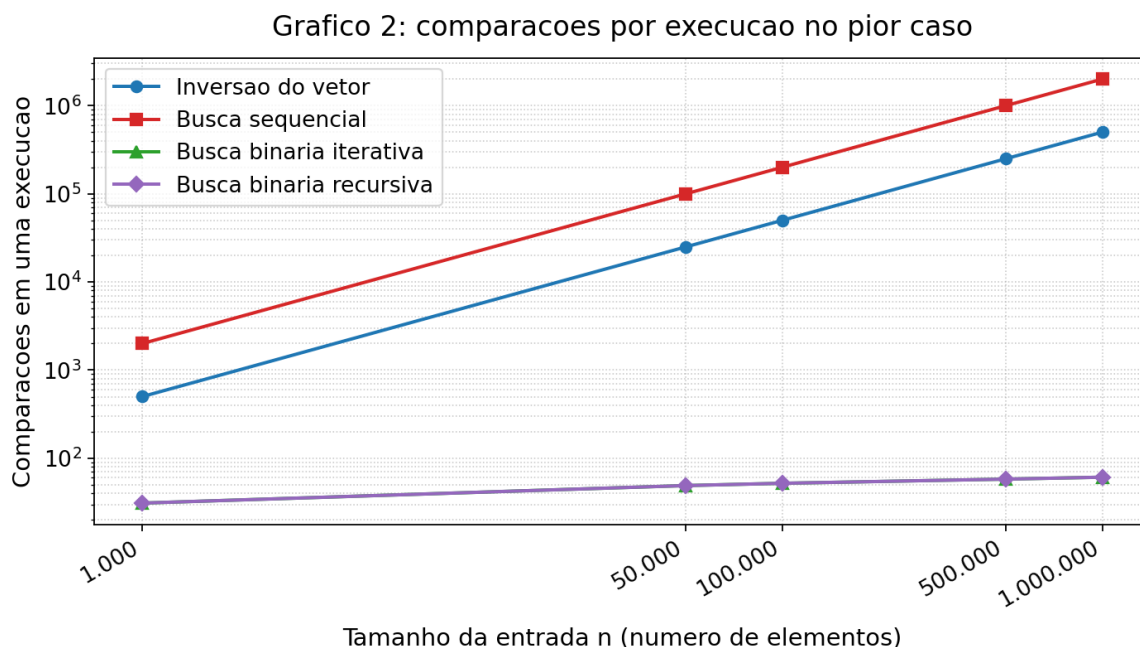
2.4.1 Gráfico 1: tempo médio dos quatro algoritmos



Os dois eixos estão em escala logarítmica. Isso é necessário porque os tempos vão de $1,3 \times 10^{-8}$ a $5,8 \times 10^{-4}$ segundos, mais de quatro ordens de grandeza: em escala linear, as duas buscas binárias ficariam indistinguíveis do eixo horizontal.

O gráfico separa os algoritmos em duas famílias. As retas inclinadas da inversão e da busca sequencial indicam crescimento proporcional a n . As retas quase horizontais das duas buscas binárias indicam um custo que praticamente não responde ao aumento da entrada.

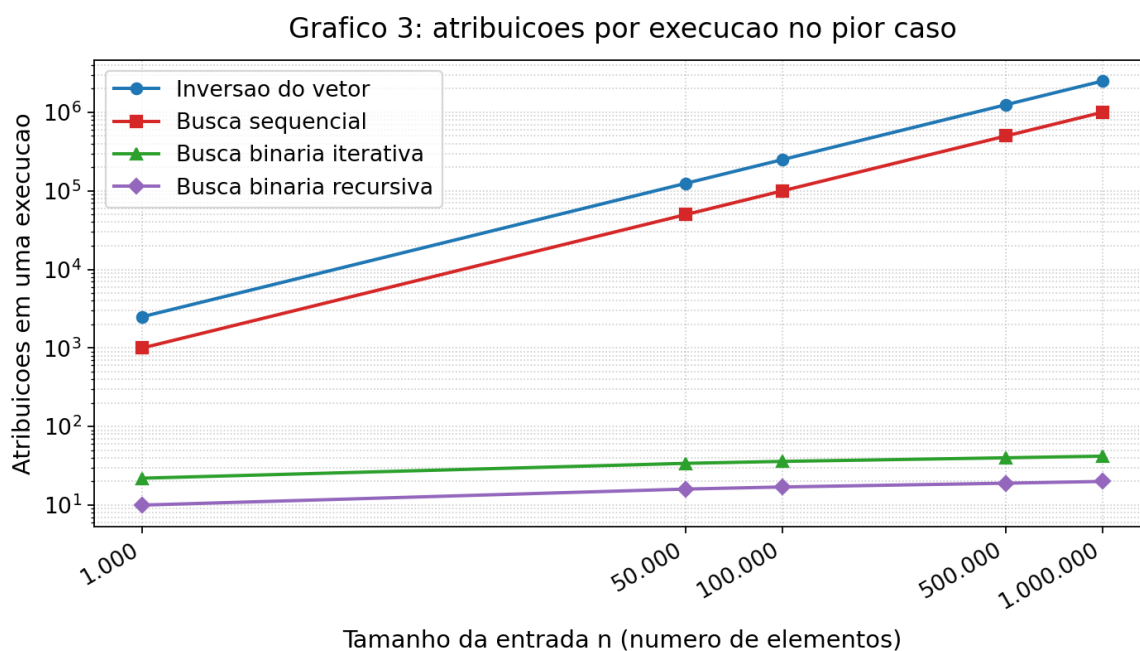
2.4.2 Gráfico 2: comparações dos quatro algoritmos



A mesma separação em duas famílias aparece aqui, agora sem nenhum ruído de medição: estes valores são contagens exatas, não medições de tempo.

A busca sequencial está acima da inversão. Isso acontece porque a busca sequencial examina n posições e faz duas comparações em cada uma, enquanto a inversão percorre apenas metade do vetor e faz uma única comparação por repetição.

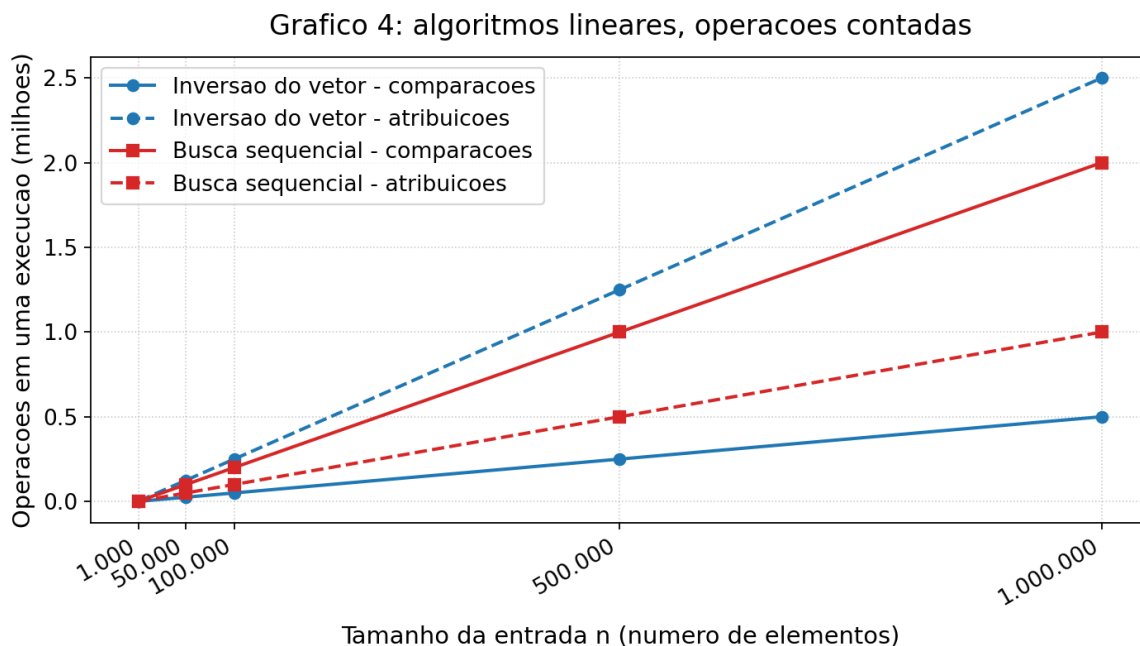
2.4.3 Gráfico 3: atribuições dos quatro algoritmos



Comparando com o Gráfico 2, as posições da inversão e da busca sequencial se invertem: em atribuições, a inversão passa a ser a curva mais alta.

Isso reflete a natureza dos dois algoritmos. A inversão faz poucas comparações e muitas trocas, cinco atribuições por repetição. A busca sequencial faz o contrário: muitas comparações e apenas uma atualização de índice por repetição.

2.4.4 Gráfico 4: algoritmos lineares em escala linear

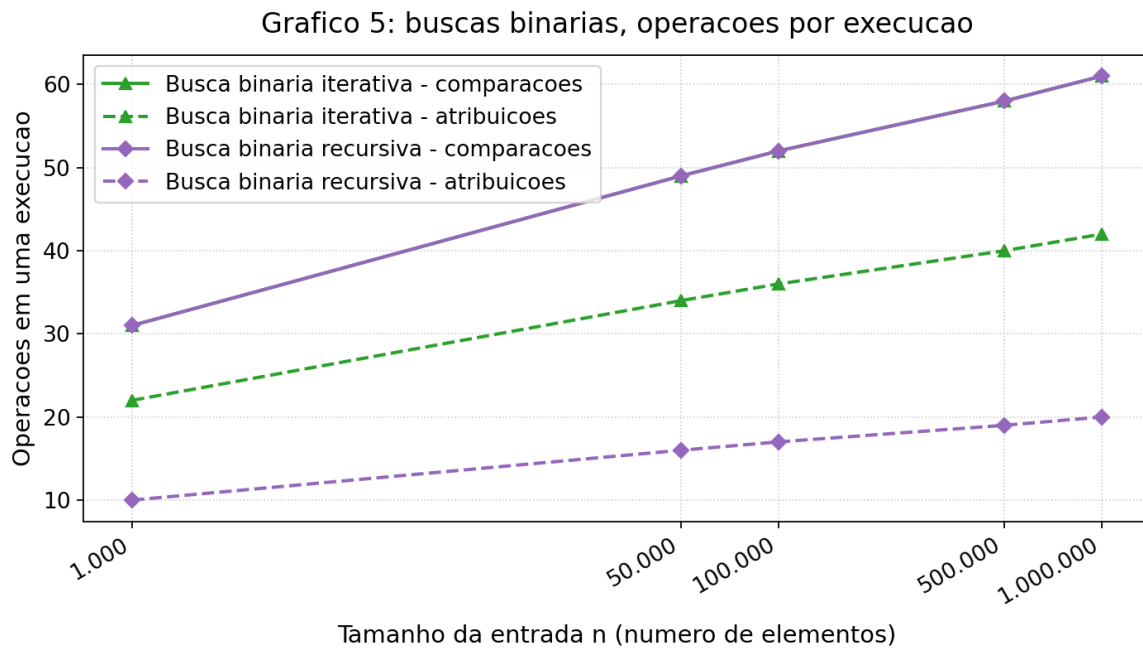


Aqui o eixo horizontal é linear, e não logarítmico. A intenção é mostrar o que a escala logarítmica esconde: as quatro séries são retas.

Ser uma reta em escala linear significa que dobrar n dobra a quantidade de

operações, exatamente o que se espera de funções de custo da forma $an + b$. É a assinatura visual do crescimento linear.

2.4.5 Gráfico 5: buscas binárias, operações em escala logarítmica



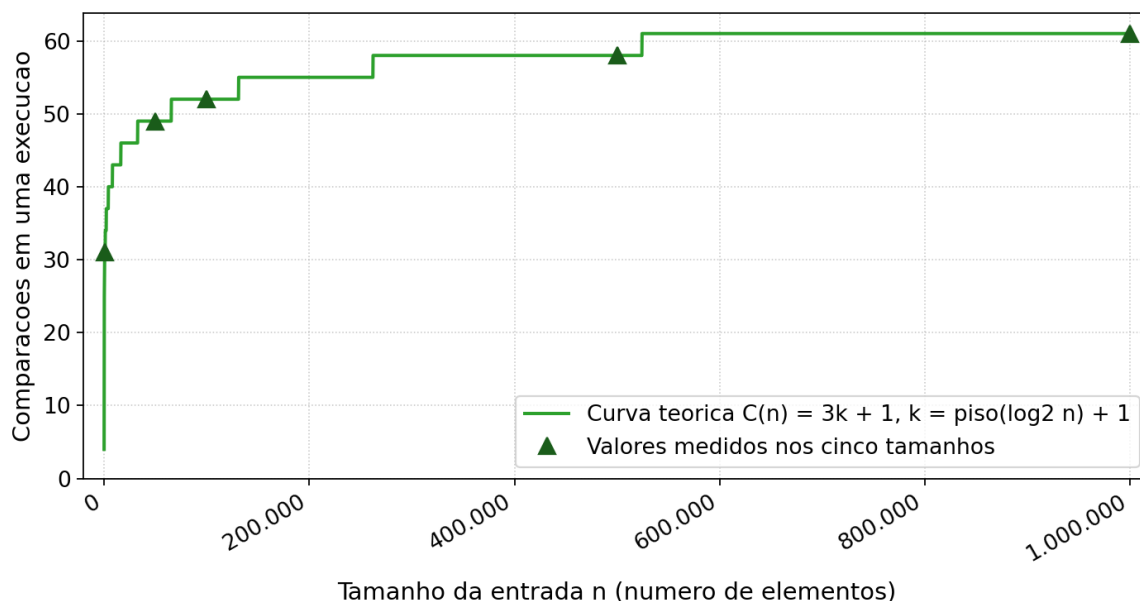
As buscas binárias precisam de um gráfico próprio: na escala dos algoritmos lineares, valores entre 10 e 61 seriam invisíveis.

O eixo horizontal é logarítmico, e as quatro séries aparecem como retas. Uma reta contra $\log n$ é a assinatura visual do crescimento logarítmico, do mesmo modo que uma reta contra n é a do crescimento linear.

As duas séries de comparações estão sobrepostas, porque as duas versões fazem exatamente o mesmo número de comparações. As séries de atribuições estão separadas, e a Seção 3.5 explica por quê.

2.4.6 Gráfico 6: busca binária, comparações em escala linear

Grafico 6: busca binaria, comparacoes em escala linear

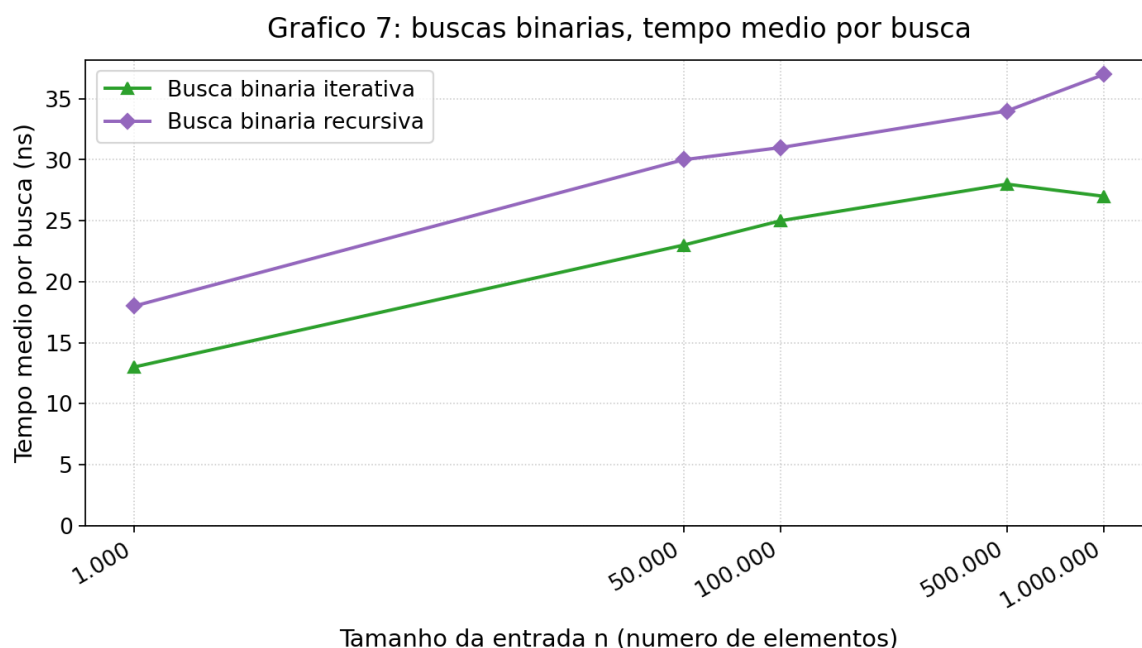


O Gráfico 5 transforma o crescimento logarítmico em uma reta, o que é útil para identificá-lo, mas esconde a forma da curva. Neste gráfico o eixo horizontal é linear, e a curva aparece como realmente é: sobe rápido para entradas pequenas e vai achatando conforme a entrada cresce.

A linha contínua é a função de custo deduzida na Seção 3.4, $C(n) = 3k + 1$ com $k = \lfloor \log_2 n \rfloor + 1$, calculada para todos os valores de n de 1 a 1.000.000. Os triângulos são os cinco valores medidos, e caem exatamente sobre ela.

Os degraus da curva são a parte mais instrutiva do gráfico. Cada degrau corresponde a uma duplicação de n : o custo só aumenta quando o vetor dobra de tamanho. Por isso o trecho entre 1 e 100.000 concentra quase toda a subida, enquanto de 500.000 até 1.000.000, um intervalo de 500.000 elementos, a curva sobe um único degrau.

2.4.7 Gráfico 7: buscas binárias, tempo médio por busca



O eixo vertical parte de zero e cobre uma faixa de poucas dezenas de nanossegundos.

A versão iterativa ficou abaixo da recursiva nos cinco tamanhos, com uma diferença entre 21% e 38%. As curvas não se cruzam em nenhum ponto.

Esse resultado é o oposto do que a contagem de operações sugere: pela contagem, a recursiva faz menos operações que a iterativa ($4k + 1$ contra $5k + 3$). A Seção 3.5 explica de onde vem essa diferença e por que ela não se traduz em menos tempo.

Uma observação sobre a leitura fina do gráfico: entre $n = 500.000$ e $n = 1.000.000$ a curva da iterativa cai de 28 para 27 ns, o que não faz sentido para um algoritmo cujo custo não diminui com a entrada. Essa queda de 1 ns corresponde exatamente a um milissegundo no tempo do bloco, que é a resolução do relógio usado, e portanto está dentro do erro de medição. Já a distância entre as duas curvas é de 5 a 10 milissegundos por bloco, bem acima dessa margem.

2.5 Discussão dos resultados

- **As contagens confirmam as fórmulas em todos os casos.** Este é o resultado mais forte da análise empírica, e também o menos sujeito a ruído. Para os vinte pares algoritmo e tamanho, as contagens medidas coincidiram exatamente com as previstas. Isso valida simultaneamente três coisas: a implementação faz o que a análise supõe, a instrumentação conta o que a convenção define, e as deduções da Seção 3 estão corretas.
- **A diferença entre $O(n)$ e $O(\log n)$ é gritante e cresce com n .** Em $n = 1.000$, a busca sequencial custa 3.002 operações contra 53 da binária iterativa, uma razão de 57 vezes. Em $n = 1.000.000$, são 3.000.002 contra 103, uma razão de 29.126 vezes. A vantagem da busca binária não é um fator constante: ela aumenta conforme a entrada cresce. Nos tempos medidos, a razão em $n = 1.000.000$ foi de cerca de 16.300 vezes, na mesma ordem de grandeza.
- **Custo contado igual não significa tempo igual.** A inversão e a busca sequencial têm custos praticamente idênticos no modelo: $3n + 3$ contra $3n + 2$.

Ainda assim, a inversão levou cerca de 30% mais tempo que a busca sequencial, e essa diferença se repetiu nos cinco tamanhos, sempre entre 29% e 33%. Uma diferença tão estável não é ruído de medição. A explicação está no que o modelo deliberadamente ignora: ele dá peso 1 a toda comparação e a toda atribuição, mas o processador não trata todas as operações com o mesmo custo. A inversão *escreve* no vetor, três atribuições por repetição ao longo de $n/2$ repetições, o que dá 1,5 milhão de escritas em memória para $n = 1.000.000$. A busca sequencial apenas *lê* o vetor e incrementa um índice, que o compilador mantém em registrador. Escrever em memória custa mais que ler, e o modelo de contagem não tem como expressar isso. Esse é um resultado importante do trabalho: o modelo descreve a ordem de crescimento, não o custo real de cada operação.

- **Nos algoritmos lineares, o tempo cresce na mesma proporção que a contagem.** Quando n passou de 1.000 para 1.000.000, ou seja, mil vezes, a contagem também cresceu cerca de mil vezes nos dois algoritmos lineares. Os tempos acompanharam de perto: 967 vezes na inversão e 978 vezes na busca sequencial. O custo por elemento ficou praticamente constante em toda a faixa, o que confirma na prática a previsão de crescimento linear. Isso não estava garantido: um vetor de um milhão de inteiros ocupa cerca de 4 MB e não cabe nas caches menores do processador. Não vimos o custo por elemento subir, provavelmente porque os dois algoritmos percorrem o vetor em ordem, que é o padrão de acesso mais fácil de o processador antecipar.
- **As buscas binárias quase não respondem ao tamanho da entrada.** Um aumento de mil vezes em n elevou o número de passos de 10 para 20 e o tempo em cerca de 2 vezes nas duas versões. É a tradução prática do fato de que $\log_2(1.000.000) \approx 20$: cada duplicação de n acrescenta um passo.
- **A versão recursiva conta menos operações e ainda assim é mais lenta.** Pela contagem, a recursiva faz $4k + 1$ operações contra $5k + 3$ da iterativa. Pelo cronômetro, a recursiva foi mais lenta nos cinco tamanhos, entre 21% e 38%. As duas medidas apontam em direções opostas, e quem está incompleto é o modelo: a convenção não conta a passagem de parâmetros, que é exatamente onde está o trabalho extra da recursão. Cada chamada precisa reservar espaço para seus parâmetros e variáveis, copiar os argumentos e, no retorno, desfazer tudo isso. Esse custo existe no processador e não aparece em nenhuma das duas colunas contadas. A conclusão prática é que, entre duas versões do mesmo algoritmo, a contagem de operações não serve para decidir qual é mais rápida; ela serve para mostrar que as duas têm a mesma ordem de crescimento, que é o que de fato se afirma sobre elas.
- **Limitações desta coleta.** Três pontos merecem registro. Primeiro, cada tempo da Seção 2.3 vem de uma única rodada de medição: não há como calcular desvio-padrão nem saber quanto o valor varia de uma execução para a seguinte. Segundo, o tempo medido inclui o custo do laço que repete as chamadas e o custo de chamar a função, e não apenas o trabalho do algoritmo. Terceiro, o relógio tem resolução de um milissegundo, o que nas buscas binárias corresponde a 1 ns por busca: diferenças dessa ordem, como a queda de 28 para 27 ns no Gráfico 7, estão dentro do erro e não devem ser interpretadas. Como o mesmo protocolo valeu para os quatro algoritmos, a comparação entre eles continua válida, e as diferenças discutidas acima são bem maiores que essa margem. Nada disso afeta as contagens, que são determinísticas e se repetem em qualquer máquina.

3 Análise Assintótica Teórica

3.1 Modelo de contagem adotado

Antes de qualquer dedução, é preciso declarar o que conta como operação. Sem essa convenção, duas pessoas analisando o mesmo código chegam a fórmulas diferentes. Adotamos:

- $C(n)$: número de comparações;
- $A(n)$: número de atribuições;
- $T(n) = C(n) + A(n)$: custo total, com peso 1 para cada operação contada.

A convenção é a seguinte:

- **Conta como comparação.** Cada avaliação de uma expressão de comparação, incluindo condições de laço e de decisão, e incluindo a avaliação final que encerra um laço. Esse último ponto é fácil de esquecer e responde pelas parcelas “+1” que aparecem nas fórmulas.
- **Conta como atribuição.** Cada inicialização com valor e cada alteração de uma variável ou posição do vetor, incluindo $i++$ e $j--$, que contam como uma atribuição cada.
- **Não é contado separadamente.** Declarações sem inicialização, operações aritméticas, acessos ao vetor, passagem de parâmetros, instruções `return`, atualizações dos próprios contadores da instrumentação e a preparação da entrada feita pelo programa auxiliar.

$T(n)$ é uma contagem de operações, não uma medida de tempo e não uma contagem de instruções de máquina. A mesma convenção foi aplicada aos quatro algoritmos, o que torna as comparações entre eles coerentes.

Uma observação sobre a notação: usamos O como limite superior, conforme pedido no enunciado, e indicamos também Θ quando a ordem de crescimento é justa nos dois sentidos. Toda classificação $\Theta(f(n))$ apresentada satisfaz igualmente $O(f(n))$.

3.2 Inversão da ordem do vetor

```
void inverte(int *v, int n) {
    int i, j, temp;
    for (i = 0, j = n - 1; i < n / 2; i++, j--) {
        temp = v[i]; v[i] = v[j]; v[j] = temp;
    }
}
```

Seja m o número de repetições do laço, igual a $n/2$ arredondado para baixo, o que se escreve $\lfloor n/2 \rfloor$. O símbolo $\lfloor \cdot \rfloor$ indica arredondamento para baixo e aparece porque, com n ímpar, o elemento central fica parado e não entra em nenhuma troca. Ocorrem duas atribuições iniciais ($i = 0$ e $j = n - 1$); a condição $i < n / 2$ é avaliada $m+1$ vezes, uma por repetição mais a final falsa; e cada repetição faz cinco atribuições, três da troca ($temp, v[i], v[j]$) e duas das atualizações ($i++, j--$).

$$C(n) = m + 1 \quad A(n) = 2 + 5m \quad T(n) = 6m + 3$$

3.2.1 Os três casos coincidem

A quantidade de operações depende apenas de n : nenhuma disposição dos valores faz o laço terminar antes ou depois. Mesmo que todos os elementos sejam iguais, as trocas são executadas. Logo, melhor caso, caso médio e pior caso têm a mesma função de custo, e o caso médio não exige hipótese de distribuição alguma, porque todas as entradas de mesmo tamanho custam o mesmo.

Para n par (todos os tamanhos do experimento), $T(n) = 3n + 3$; para n ímpar, $T(n) = 3n$.

3.2.2 Termo dominante e complexidade

O termo dominante é $3n$, linear. O fator constante e a parcela aditiva não alteram a ordem de crescimento; percorrer apenas metade do vetor reduz o custo por um fator 2, o que também não muda a classificação.

Caso	$T(n)$	Classificação
Melhor	$6\lfloor n/2 \rfloor + 3$	$\Theta(n)$, logo $O(n)$
Médio	$6\lfloor n/2 \rfloor + 3$	$\Theta(n)$, logo $O(n)$
Pior	$6\lfloor n/2 \rfloor + 3$	$\Theta(n)$, logo $O(n)$

Além do vetor que recebe, a função usa apenas as variáveis i , j e temp . Essa quantidade é sempre a mesma, não importa o tamanho do vetor, então a memória extra que o algoritmo consome é constante, $O(1)$. A cópia que o programa principal faz antes de inverter ocupa memória proporcional a n , mas é feita fora da função analisada.

3.3 Busca sequencial

```
int busca_sequencial(int *v, int n, int valor) {
    int i;
    for (i = 0; i < n; i++)
        if (v[i] == valor) return 1;
    return 0;
}
```

Chamando de j o índice, contado a partir de zero, em que o valor foi encontrado, e distinguindo o término por sucesso do término por esgotamento do vetor:

Situação	Posições examinadas	C	A	T
Encontrado em j	$j + 1$	$2(j + 1)$	$j + 1$	$3(j + 1)$
Ausente	n	$2n + 1$	$n + 1$	$3n + 2$

A diferença entre as duas linhas merece atenção: quando o valor é encontrado, a função retorna de dentro do laço, e nem o $i++$ daquela repetição nem a reavaliação de $i < n$ chegam a acontecer. Quando o valor está ausente, o laço só termina com a avaliação final falsa de $i < n$, que é contada.

3.3.1 Melhor caso

Valor na primeira posição ($j = 0$): $C = 2$, $A = 1$, $T = 3$. Custo constante, $\Theta(1)$.

3.3.2 Caso médio

Hipóteses: os elementos são distintos, o valor procurado está no vetor, e qualquer uma das n posições tem a mesma chance de ser a certa.

Se o valor está na primeira posição, a busca examina uma posição; se está na segunda, examina duas; e assim por diante, até n posições quando o valor está na última. Os valores possíveis vão de 1 a n , de um em um, e todos têm a mesma chance. A média de uma sequência assim é o valor do meio, isto é, a média entre o primeiro e o último:

$$\text{posições examinadas, em média} = \frac{1 + n}{2}$$

Ou seja, em média a busca percorre metade do vetor. Substituindo esse valor nas fórmulas da tabela anterior:

$$C_{\text{médio}}(n) = n + 1 \quad A_{\text{médio}}(n) = \frac{n + 1}{2} \quad T_{\text{médio}}(n) = \frac{3(n + 1)}{2}$$

Se incluíssemos também as buscas que falham, o custo médio ficaria em algum ponto entre $\frac{3(n+1)}{2}$, quando nenhuma falha, e $3n + 2$, quando todas falham. Em qualquer proporção entre as duas, o crescimento continua proporcional a n .

3.3.3 Pior caso

O laço executa n vezes. A condição $i < n$ é avaliada $n + 1$ vezes (uma por repetição mais a final falsa) e $v[i] == \text{valor}$ é avaliada n vezes:

$$C(n) = (n + 1) + n = 2n + 1 \quad A(n) = 1 + n = n + 1$$

$$T(n) = 3n + 2$$

3.3.4 Termo dominante e complexidade

O termo dominante é $3n$, linear.

Caso	$T(n)$	Classificação
Melhor	3	$\Theta(1)$, logo $O(1)$
Médio	$3(n + 1)/2$	$\Theta(n)$, logo $O(n)$
Pior	$3n + 2$	$\Theta(n)$, logo $O(n)$

Examinar metade do vetor em média reduz o custo por um fator constante, mas não a ordem de crescimento, exatamente o tipo de diferença que a notação assintótica descarta.

Além do vetor, a função usa apenas a variável i , então a memória extra é constante, $O(1)$.

3.4 Busca binária iterativa

```
int busca_binaria_iterativa(int *v, int n, int valor) {
    int inicio = 0, fim = n - 1, meio;
    while (inicio <= fim) {
        meio = inicio + (fim - inicio) / 2;
        if (v[meio] == valor) return 1;
        if (v[meio] < valor) inicio = meio + 1;
        else fim = meio - 1;
    }
    return 0;
}
```

Uma repetição com intervalo não vazio em que o elemento central não é o procurado faz três comparações ($\text{inicio} \leq \text{fim}$, $v[\text{meio}] == \text{valor}$, $v[\text{meio}] < \text{valor}$) e duas atribuições (meio e a atualização de um dos limites, sendo que exatamente uma das duas é executada). Se o elemento for encontrado, a repetição faz duas comparações e uma atribuição. Se o intervalo ficar vazio, ocorre apenas a comparação $\text{inicio} \leq \text{fim}$, falsa.

3.4.1 Melhor caso

Valor no meio do intervalo inicial: $C = 2$, $A = 3$ (as duas inicializações mais meio), $T = 5$. Constante, $\Theta(1)$.

3.4.2 Caso médio

Mesmas hipóteses da busca sequencial. Chamando de d o número de repetições até encontrar o elemento, as primeiras $d - 1$ repetições fazem 3 comparações e 2 atribuições cada, e a última faz 2 comparações e 1 atribuição. Somando as duas inicializações:

$$C = 3d - 1 \quad A = 2d + 1 \quad T = 5d$$

Falta saber quanto vale d na média. Para isso, ajuda pensar na busca por níveis. No primeiro nível está um único elemento, o do meio do vetor, encontrado em uma repetição. No segundo nível estão no máximo dois elementos, encontrados em duas repetições. No terceiro, no máximo quatro. O número de elementos dobra a cada nível, e o último nível é alcançado em k repetições, o mesmo k do pior caso.

Como a quantidade dobra a cada nível, o último nível sozinho contém mais da metade dos elementos do vetor. Isso significa que, para mais da metade dos valores que se pode procurar, a busca gasta exatamente k repetições, e para os demais gasta menos. A média fica portanto um pouco abaixo de k e cresce junto com ele.

Como k é proporcional a $\log n$, o caso médio também é, ou seja, $\Theta(\log n)$.

3.4.3 Pior caso

Seja k o número de repetições com intervalo não vazio. Cada repetição deixa um intervalo de $\lfloor n/2 \rfloor$ elementos; após k reduções o tamanho é $\lfloor n/2^k \rfloor$, e o laço termina quando esse valor chega a zero, isto é, quando $2^k > n$. Para $n \geq 1$, o menor inteiro que satisfaz isso é

$$k = \lfloor \log_2 n \rfloor + 1$$

Somando as k repetições (3 comparações e 2 atribuições cada), a comparação final falsa e as duas inicializações:

$$C(n) = 3k + 1 \quad A(n) = 2k + 2$$

$$T(n) = 5k + 3 = 5\lfloor \log_2 n \rfloor + 8$$

Em forma de recorrência, com as inicializações contadas à parte: $T(0) = 1$ e $T(n) = 5 + T(\lfloor n/2 \rfloor)$.

Caso	$T(n)$	Classificação
Melhor	5	$\Theta(1)$, logo $O(1)$
Médio	proporcional a $\log n$	$\Theta(\log n)$, logo $O(\log n)$
Pior	$5\lfloor \log_2 n \rfloor + 8$	$\Theta(\log n)$, logo $O(\log n)$

A base do logaritmo não altera a classificação, pois mudar de base multiplica a função por uma constante. Usamos base 2 porque o intervalo é dividido ao meio.

Além do vetor, a função usa apenas as variáveis *inicio*, *fim* e *meio*, então a memória extra é constante, $O(1)$.

3.5 Busca binária recursiva

```
int busca_binaria_recursiva(int *v, int inicio, int fim, int p) {
    if (inicio > fim) return 0;
    int meio = inicio + (fim - inicio) / 2;
    if (v[meio] == p) return 1;
    if (v[meio] < p) return busca_binaria_recursiva(v, meio + 1, fim, p);
    else return busca_binaria_recursiva(v, inicio, meio - 1, p);
}
```

Uma chamada com intervalo não vazio em que o elemento central não é o procurado faz três comparações ($\text{inicio} > \text{fim}$, $v[\text{meio}] == p$, $v[\text{meio}] < p$) e uma atribuição (*meio*). A chamada final com intervalo vazio faz apenas a comparação $\text{inicio} > \text{fim}$ e nenhuma atribuição. A passagem dos novos limites como parâmetros não é contada como atribuição, conforme a convenção.

3.5.1 Melhor caso

Valor no meio do intervalo inicial: $C = 2$, $A = 1$, $T = 3$. Constante, $\Theta(1)$.

3.5.2 Caso médio

Mesmas hipóteses. Com d chamadas até encontrar o elemento: $C = 3d - 1$, $A = d$ e $T = 4d - 1$.

A versão recursiva visita exatamente a mesma sequência de elementos centrais que a iterativa, então a média de d é a mesma das duas, e o raciocínio por níveis da Seção 3.4 vale sem mudança. O caso médio é portanto $\Theta(\log n)$.

3.5.3 Pior caso

As recorrências, com $k = \lfloor \log_2 n \rfloor + 1$ chamadas não vazias mais a chamada final vazia:

$$C(0) = 1, \quad C(n) = 3 + C(\lfloor n/2 \rfloor) \quad A(0) = 0, \quad A(n) = 1 + A(\lfloor n/2 \rfloor)$$

$$T(0) = 1, \quad T(n) = 4 + T(\lfloor n/2 \rfloor)$$

Desdobrando até o caso-base:

$$C(n) = 3k + 1 \quad A(n) = k$$

$$T(n) = 4k + 1 = 4\lfloor \log_2 n \rfloor + 5$$

Caso	$T(n)$	Classificação
Melhor	3	$\Theta(1)$, logo $O(1)$
Médio	proporcional a $\log n$	$\Theta(\log n)$, logo $O(\log n)$
Pior	$4\lfloor \log_2 n \rfloor + 5$	$\Theta(\log n)$, logo $O(\log n)$

3.5.4 Comparação com a versão iterativa

As duas percorrem o mesmo caminho e fazem o mesmo número k de passos não vazios, de modo que as comparações são idênticas. A diferença está nas atribuições:

Medida no pior caso	Iterativa	Recursiva
$C(n)$	$3k + 1$	$3k + 1$
$A(n)$	$2k + 2$	k
$T(n)$	$5k + 3$	$4k + 1$

A iterativa conta mais atribuições porque atualiza explicitamente início ou fim a cada repetição, além das duas inicializações; na recursiva, os novos limites viajam como parâmetros, que a convenção não conta. Isso é uma consequência do modelo de contagem, não uma vantagem prática da recursão: o modelo não cobre o custo de criar e destruir os blocos de memória de cada chamada recursiva. Os tempos medidos confirmam: a recursiva executa menos operações contadas e ainda assim foi mais lenta nos cinco tamanhos, como mostra o Gráfico 7.

Há ainda uma diferença que a contagem de operações não captura: a memória extra que cada versão consome.

A versão iterativa trabalha sempre com as mesmas três variáveis, então sua memória extra é constante. A recursiva, não: enquanto uma chamada espera o

resultado da chamada seguinte, ela continua ocupando espaço com seus próprios parâmetros e variáveis. Com k chamadas encadeadas, são k blocos ocupados ao mesmo tempo, e a memória extra cresce proporcionalmente a $\log n$.

Para $n = 10^6$ isso significa 21 blocos, o que é irrelevante na prática, mas é uma diferença real entre as duas versões. Em vetores muito maiores, ou em algoritmos recursivos mais profundos, é o tipo de custo que pode esgotar a memória reservada para as chamadas.

3.6 Quadro-resumo

Algoritmo	Melhor caso	Caso médio [†]	Pior caso	Memória extra
Inversão do vetor	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
Busca sequencial	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
Busca binária iterativa	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(1)$
Busca binária recursiva	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$

[†] Para as três buscas, o caso médio supõe busca bem-sucedida em vetor de elementos distintos, com cada uma das n chaves igualmente provável. A inversão independe dos valores, então seu caso médio não requer hipótese nenhuma. Toda classificação $\Theta(f(n))$ acima satisfaz igualmente $O(f(n))$.

4 Conclusão

Os gráficos separam os quatro algoritmos em duas famílias, e a separação é a prevista pela teoria. A inversão percorre metade do vetor e a busca sequencial percorre o vetor inteiro: as duas crescem proporcionalmente a n e aparecem nos gráficos como retas inclinadas. A busca binária iterativa e a busca binária recursiva descartam metade do intervalo a cada passo: as duas crescem proporcionalmente a $\log n$ e aparecem como retas quase horizontais. Essa leitura vale tanto para as contagens exatas quanto para os tempos medidos, apesar do ruído inerente à medição.

A busca binária não é apenas mais rápida que a busca sequencial: a diferença entre as duas aumenta conforme o vetor cresce. Em $n = 1.000$, a busca sequencial faz 3.002 operações e a binária iterativa faz 53, ou seja, 57 vezes mais. Em $n = 1.000.000$, são 3.000.002 contra 103, ou seja, 29.126 vezes mais. O vetor ficou mil vezes maior e a vantagem da busca binária ficou cerca de 500 vezes maior. É isso que significa dizer que $O(\log n)$ é melhor que $O(n)$: a diferença entre os dois não é um número fixo, ela cresce junto com a entrada. Nos tempos medidos aparece o mesmo efeito: em $n = 1.000.000$, a busca sequencial levou cerca de 440 μs e a binária iterativa cerca de 27 ns, uma razão de aproximadamente 16.300 vezes.

Entre as duas versões da busca binária, a contagem e o cronômetro discordam, e isso é instrutivo. Pela contagem, a recursiva faz menos operações: $4k + 1$ contra $5k + 3$. A diferença aparece porque a iterativa atualiza início ou fim a cada passo, enquanto a recursiva passa os novos limites como parâmetro, e a convenção adotada não conta passagem de parâmetro como atribuição. Pelo cronômetro, no entanto, a recursiva foi mais lenta nos cinco tamanhos, entre 21% e 38%, sem que as curvas se cruzem em nenhum ponto. Quem está incompleto é o modelo: aquilo que ele deixa de contar, a passagem de parâmetros, é justamente o trabalho extra da recursão, porque cada chamada precisa reservar espaço para seus dados, copiar os argumentos e desfazer tudo no retorno. A diferença de memória vai na mesma direção: a iterativa usa sempre as mesmas três variáveis, enquanto a recursiva mantém um bloco ocupado por cada chamada que ainda não terminou, ou seja, $O(1)$ contra $O(\log n)$. Entre duas versões do mesmo algoritmo, portanto, a contagem de operações não decide qual é mais rápida; ela mostra que as duas têm a mesma ordem de crescimento, que é o que de fato se afirma sobre elas.

A busca binária, porém, só funciona com o vetor ordenado. A busca sequencial é a única das três que funciona em um vetor qualquer, e é isso que ela troca por seu custo maior. Neste trabalho o vetor já vinha ordenado por exigência do enunciado, e o custo de ordená-lo ficou fora de todas as medições. Se o vetor não estivesse ordenado e fosse consultado uma única vez, ordenar para depois usar busca binária sairia mais caro do que simplesmente varrer o vetor.

O trabalho também mostra os limites do modelo de contagem, e o par inversão e busca sequencial é o exemplo mais claro. Os dois fazem quase o mesmo número de operações ($3n + 3$ e $3n + 2$), mas a inversão levou cerca de 30% mais tempo, de forma estável nos cinco tamanhos. O motivo é que a inversão escreve no vetor três vezes por repetição, enquanto a busca apenas lê e incrementa um índice, e escrever em memória custa mais que ler. O modelo dá peso 1 às duas coisas e não tem como distinguir uma da outra. Nada disso contraria as classificações obtidas. O modelo conta operações e diz como o custo cresce, não quanto cada operação custa em segundos numa máquina real. Por isso a classificação vem da dedução, e os gráficos servem para conferir se o comportamento previsto aparece.

5 Contribuições dos integrantes

Integrante	nº USP	Principal contribuição
Enzo Ribeiro Ferrari	15574520	Inversão do vetor: implementação, programa auxiliar, coleta, análise teórica e integração do programa principal, organização do relatório e do projeto.
Matheus de Souza Santana	17839560	Busca sequencial: implementação, programa auxiliar, coleta e análise teórica.
Pietro Augusto Venancio da Costa	17839104	Busca binária iterativa: implementação, programa auxiliar, coleta e análise teórica.
Matheus Bastos Machado	18126601	Busca binária recursiva: implementação, programa auxiliar, coleta, análise teórica, integração do programa principal, organização do relatório e do projeto.

6 Considerações finais

6.1 Organização do projeto e repositório

O trabalho foi desenvolvido em um repositório no GitHub, compartilhado pelos quatro integrantes, com todo o histórico de alterações versionado. Trabalhamos diretamente na branch principal, em commits pequenos, o que exigiu combinar antes quem estava editando cada arquivo, já que as funções dos quatro algoritmos ficam em um arquivo comum.

Dentro do repositório, separamos o projeto em pastas por finalidade, e essa organização foi o que tornou o trabalho divisível entre quatro pessoas:

Pasta	Conteúdo
src/	Programa principal e as funções dos quatro algoritmos, sem contadores.
experiments/	Um programa auxiliar por algoritmo, com a versão instrumentada e a rotina de medição.
data/	Resultados em CSV e os metadados do ambiente de medição.
analysis/	Dedução detalhada da função de custo de cada algoritmo.
graphs/	Gráficos e o script que os gera a partir dos CSVs.
report/	Este relatório.

A separação entre `src/` e `experiments/` foi a decisão de organização mais importante do trabalho. Ela é o que garante, na prática, que os contadores nunca entrem na função cronometrada: a função medida é sempre a de `src/`, e a versão instrumentada existe apenas dentro do programa auxiliar. Sem essa separação física dos arquivos, seria fácil misturar as duas coisas sem perceber.

Cada integrante ficou responsável pelo ciclo completo do seu algoritmo: implementação, programa auxiliar, coleta dos dados, análise teórica e o texto correspondente. Os gráficos e a montagem final do relatório foram organizados em conjunto.

6.2 Dificuldades encontradas

A implementação dos quatro algoritmos foi a parte mais tranquila do trabalho. As dificuldades apareceram nas etapas seguintes:

- **Análise teórica.** Foi mais trabalhosa do que esperávamos. Contar operações parece simples até o momento em que é preciso decidir, por exemplo, se a avaliação final que encerra o laço conta ou não, e o que acontece com o `i++` da repetição em que a função retorna. Essas decisões mudam as fórmulas, e como os quatro algoritmos precisavam seguir a mesma convenção, qualquer mudança obrigava a revisar tudo de novo.
- **Gráficos.** Também deram trabalho, principalmente por causa da escala. Os valores cobrem cinco ordens de grandeza: em escala linear, as duas buscas binárias simplesmente desaparecem no eixo horizontal. Foi preciso entender quando usar escala logarítmica, quando ela esconde informação, e por que alguns gráficos precisavam ser separados.
- **Códigos auxiliares.** Foram a maior dificuldade. Escrever um programa que mede tempo de forma confiável envolve questões que não aparecem em um exercício comum: cronometrar um bloco de repetições em vez de uma execução isolada, manter alocação e preenchimento fora da região medida, impedir que o compilador elimine a chamada por considerar o resultado inútil, e separar a contagem da medição de tempo.

Nessas três partes, usamos uma ferramenta de inteligência artificial (Claude, da Anthropic) como apoio: para revisar as deduções e as convenções de contagem, para decidir a apresentação dos gráficos e, principalmente, para estruturar os programas auxiliares de medição. A redação deste relatório contou com o mesmo apoio, na organização das seções, na formatação das tabelas e na revisão do texto. O uso foi de auxílio e revisão.